Machine Learning For Networking

Stridi Simone

Anno Accademico 2026/2027

Contents

Chapter 1

Model Selection and Validation

The $\mathcal{H}$ set of hypothesis can be nested, as polynomial hypothesis h can be built from linear hypothesis h. If we consider a polynomial hypothesis h of degree d at a certain point we will reach ERM 0, as we can perfectly fit any m data points using polynomials of degree n as soon as $n \geq m - 1$. However, it is not a best practice to pursue since it leads to **overfitting**.

Overfitting happens when a model performs really well on the training data, as the case for high degree polynomials h, but has poor performance on the unseen test data. This means that a small train error does not guarantee good performance outside the training set, on new data points. Instead, small training error only indicates that we have solved the ERM optimization problem, hence our model correctly minimizes the ERM. Just as a reminder, on the contrary, underfitting occurs when a model is too simple and fails to capture the underlying relationship in the data, it is characterized by poor performance in the training error.

It is like our trained model h function is so fixed on the training data that any new data different from it may be badly classified.
Since as we have seen only considering the train error can mislead, the concept of Validation is needed.

1.1 Validation

The basic idea behind Validation is to divide the initial dataset into two subsets:

- Training and Validation Set.
- Use Training Set to learn a hypothesis h
- Use Validation Set to probe outside the training set in order to estimate the expected loss.

In this way, h will still learn on the training set but the ERM performance will be assessed on the Validation Data. Therefore, we will choose our model via the validation error. Firstly, the ERM is performed on the training set, then it is also assessed on the validation error. The end-goal is to select the model that minimizes the training error and has the smallest validation error.

- Overfitting: model performs much better on the training set than on the validation set.

- Underfitting: more or less same performance, but really low since the model is not complex enough to learn the best h.

It may happen that when splitting into training and validation set we encounter an unlucky split, leading to non optimal results. To tackle this issue, **k-fold cross validation** is exploited.

1.2 K-Fold Cross Validation

Since we might be unlucky with the train-val split and this is really problematic for small datasets, the concept of K-Fold Cross Validation is to randomly split train-val several times, in order to average out the unlucky splits. The splits will be performed K times.

In K-Fold Cross Validation the training error and the validation error are the average empirical loss over the k training folds. In order to optimally select the K number of folds, the train fold should be sufficiently large to avoid overfitting and the validation folds should be sufficiently large to get reliable estimate of generalization. Warning: K-Fold works if the data is i.i.d. and the order of data points is arbitrary, however in the case of grouped or ordered data points this is not true, hence k-fold might fail in this case.
For unbalanced datasets in classes or groups the StratifiedKFold and GroupKFold functions are needed.

1.2.1 Test Set

Additionally, we need a test set different from training and validation set, since the real valid error might not be that optimistic anymore, as validation error is a poor performance indication. There might be some noise, it might be too optimistic since it is chosen with smallest loss.

1.2.2 Model Selection Recipe

- Input: list of candidate models
- For each candidate model:
  1. Learn optimal h by minimizing the training error.
  2. Compute the validation error on the validation set.
- Choose h with minimum validation error.
- Compute test error on test set.

To sum up, ML methods using large models tend to overfit. It is needed to probe the learnt hypothesis outside the training set by validating it on the validation set. Pick the model with minimum validation error. Finally, compute the test error for the chosen model.

Chapter 2

ML Metrics and HyperParameter Tuning

2.1 Performance Metrics

Losses are for machines, while it is needed to define metrics for humans. Metrics are a measure of quality and can be exploited in order to choose the hyper-parameters of a model.

2.1.1 Evaluation of supervised ML

- Efficiency: model building time, prediction time.
- Scalability: training set size, attribute number.
- Robustness: noise, missing data.
- Interpretability: model interpretability, model compactness.

These are independent from the correctness of the resulting y' labels.

2.1.2 Evaluation of Regression Techniques

The quality of the prediction is assessed through loss functions such as mean squared error, mean absolute error and other ones.

2.2 Evaluation of Classification Techniques

The quality of the prediction can be assessed through:

- Confusion Matrix.
- Accuracy.
- Precision, Recall, F-measure per class and on average.
- ROC curve and other metrics.

2.2.1 Confusion Matrix

The Confusion Matrix is used to evaluate a classification task implemented by binary classifiers. It is built as follows:

- Rows: Actual Class
- Columns: Predicted Class
- TP: True Positives when $y = y'$ for the positive class.
- TN: True Negatives when $y = y'$ for the negative class.
- FP: False Positives when y' is positive while y is negative.
- FN: False Negatives when y' is negative while y is positive.

Actual \ Predicted	**P**	**N**
P	True Positives (TP)	False Negatives (FN)
N	False Positives (FP)	True Negatives (TN)

Table 2.1: Confusion Matrix

It can go up to $(n \times n)$. The correctly predicted values are always on the diagonal.

2.2.2 Accuracy

Accuracy is the most widely-used metric for model evaluation and it is computed as follows:

$$\text{Accuracy} = \frac{\text{Number of correctly classified objects}}{\text{Number of classified objects}}$$

For binary classifiers:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

In addition accuracy has the following properties:

- Accuracy is related to the 0/1 loss: it goes as $1-$ (0/1 loss value).
- Accuracy might be misleading in case of unbalanced classes or different class relevance and importance. Therefore, specific class measures are defined.

2.2.3 Class Specific Measures

Class specific measures need to be evaluated separately for each class c. They are the precision p, the recall r and the F-measure, which is the harmonic mean of the precision and recall. They are defined as follows:

$$\text{Precision}(c) \quad p = \frac{\text{Number of objects correctly assigned to } c}{\text{Number of objects assigned to } c}$$

$$\text{Recall}(c) \quad r = \frac{\text{Number of objects correctly assigned to } c}{\text{Number of objects belonging to } c}$$

$$\text{F-measure}(c) \quad F = \frac{2rp}{r + p}$$

For a binary classification problem the following are the simplified formulas for the positive class c:

$$\text{Precision} \quad p = \frac{TP}{TP + FP}$$

$$\text{Recall (True Positive Rate)} \quad r = \frac{TP}{TP + FN}$$

$$\text{F-measure} \quad F = \frac{2rp}{r + p} = \frac{2TP}{2TP + FP + FN}$$

2.2.4 ROC Curve

The Receiver Operating Characteristic (ROC) was developed for analyzing noisy signals and finding the trade-off between positive hits and false alarms. The ROC curve shows how the FP and TP rates change when changing the decision threshold. In $(0, 0)$ we declare everything to be negative class, while in $(1, 1)$ everything to be positive class. The diagonal line is random guessing if classes are balanced; below the diagonal line the prediction is opposite of the true class.
It is important to compute the area under the curve (AUC) and choose the model with the largest area.

M1 is better for small FPR, while M2 is better for large FPR.

How to build ROC curve?

- Use a classifier that produces the posterior probability $P(+|x)$ for each sample x.
- Apply a threshold at each unique value of $P(+|x)$.
- Compute TPR and FPR at each threshold.

Since in the example we have 10 samples, $m = 10$ columns. We need to order them according to probability horizontally.

2.2.5 Learning Curve

The Learning Curve shows how a metric, or loss, changes with varying training sample size. It is possible to use different losses for training and validation, and this enables the comparison of different ML methods. It is a tool to find out how much we benefit from adding more training data.

2.3 HyperParameter Tuning

Hyper-Parameters are parameters that are **not directly learnt within estimators**. **Hyper-Parameters define the model space** and they are passed as arguments to the constructor of the estimator classes in scikit-learn. The aim is to search the hyper-parameters space for the best cross-validation score metric. Hence, hyper-parameters are really important for the final model selection.
The **Validation Curve** can be used to assess the influence of a single hyper-parameter on the training score and the validation score. Hence, it is possible to find out whether the estimator is overfitting or underfitting for some hyper-parameter values.

2.3.1 Grid Search

Grid Search exhaustively generates candidates from a **grid of hyper-parameter values**. A different hypothesis is trained for all possible combinations of hyper-parameter values.

2.4 Recap

To sum up, these are the steps to follow when building an ML process:

- Load/generate datasets.
- Choose samples, features and labels.
- Split the dataset in training, validation (or later cv) and testing.
- Pre-processing techniques.
- Define model and loss.
- Perform ERM with an algorithm (learn).
- Evaluate the obtained performance (metrics).
- Tune the hyper-parameters.
- Test the final hypothesis on test data.
- Use it in the wild for inference.